

```

    fmt.Printf(" ")
    return nil;
} else {
    return errors.New("Show:: The stack is empty")
}
}

```

We have changed our *main* accordingly. This time, we have used multiple assignment notations to copy the data as well as error value returned from *Pop()*.

```

$GOPATH/src/gostack/gostack.go
// the code in this file belongs to the 'main' package
package main
// import our 'stack' and other required packages
import (
    "gostack/stack"
    "fmt"
)
// the 'main' function, entry point of our program
func main() {
    var myStack stack.Stack

    // show the empty stack
    if myStack.Show()!=nil {
        fmt.Println("Show:: The stack is empty")
    }
    // try to pop from the empty stack
    if item,err:=myStack.Pop(); err==nil {
        fmt.Printf("\nThe Popped item=%v\n",item)
    }else{
        fmt.Println("Pop:: The stack is empty")
    }
    // push 10,20 into the stack
    myStack.Push(10)
    myStack.Push(20)
    // show the stack
    myStack.Show()

    // pop from the stack
    if item,err:=myStack.Pop(); err==nil {
        fmt.Printf("\nThe Popped item=%v\n",item)
    }else{
        fmt.Println("Pop:: The stack is empty")
    }
    // show the stack
    if myStack.Show()!=nil {
        fmt.Println("Show:: The stack is empty")
    }
}

```

Now, it's time to add exception handling to our program. We have modified our *Pop()* method to demonstrate how exception handling works. We have

panic() and *recover()* built-in functions. A detailed description of these functions is beyond the scope of this article. We will only describe our own use case here. If *Pop()* method is called on an empty stack, we generate a panic with a user-given message. By default, *panic* gets propagated through the call stack of functions. But, we want to capture it in the same function and handle it. At the beginning of the *Pop()* we have called an anonymous function, i.e., a function without name. But this doesn't get executed immediately because of the *defer* keyword. This is called a deferred function, which gets executed at the end of the function before returning to the caller. That means that just before leaving the *Pop()* function, we check if any panic has been generated, and if so, we print the panic message to the console.

```

$GOPATH/src/gostack/stack/stack.go
// 'Pop' function to pop item from the stack given as
receiver
func (stack *Stack) Pop() (item interface{},e error) {
    e=errors.New("Empty stack")
    defer func() {
        if ret:=recover(); ret!=nil {
            fmt.Printf("%v\n",ret)
        }
    }()

    if len(*stack)==0 { // len() returns the no of elements
in a slice
        // call panic() with the error msg
        panic("Pop:: The stack is empty")
    }
    // remove the last element from the slice in item
    item=(*stack)[len(*stack)-1]
    // creates a new slice by taking the ele from first to
second last
    *stack=(*stack)[:len(*stack)-1]
    return item,nil
}

```

In the next article on Go, we will discuss concurrency support along with a suite to test a Go program. 

References

- [1] 'An Introduction To Programming in Go' by Caleb Doxsey
- [2] 'Programming in Go' by Mark Summerfield
- [3] <https://golang.org>

By: Ramaprasad Chakraborty

The author works at ISRO, Sriharikota, as a scientist engineer SD and has six years of industry experience. He is interested in real-time operating systems, controller redundancy mechanisms, networking, cyber forensics, database management systems and task automation with different levels of scripting and various open source technologies.